

第15周实验指导书：Spring Security与JWT安全认证集成

实验基本信息

课程名称：高级程序设计与系统开发工具

实验章节：第15周 用户注册、登录与安全

实验学时：4学时

实验类型：综合型实验

实验分组：个人独立完成

前置知识：Week14多用户架构、Spring Boot基础、Maven依赖管理

实验目的与要求

1. 实验目的

- 掌握Spring Security集成**：理解Spring Security的核心过滤器链机制，学会配置基本的安全策略。
- 掌握JWT认证机制**：实现基于Token的无状态认证，包括Token的生成、解析与校验。
- 实现安全的用户管理**：掌握密码的加密存储（BCrypt）与验证。
- 重构用户上下文**：将第14周的简易 ThreadLocal 上下文升级为基于 SecurityContext 的标准安全上下文。

2. 实验要求

- 引入安全框架**：在项目中集成Spring Security和JWT库。
- 实现注册登录**：开发 /api/auth/register 和 /api/auth/login 接口。
- 保护业务接口**：确保只有持有有效Token的用户才能访问笔记API。
- 数据隔离升级**：确保Service层能从安全上下文中正确获取当前登录用户的ID。

实验环境与工具

- JDK：17+
- IDE：IntelliJ IDEA / Eclipse / VS Code

- **构建工具**：Maven
- **调试工具**：Postman (必须，用于测试API和Header)

实验内容与步骤

第一部分：环境搭建与依赖引入 (0.5学时)

任务 1-1: 添加Maven依赖

在 `pom.xml` 中添加 Spring Security 和 JWT 相关的依赖。

```
<!-- Spring Security 核心 -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<!-- JWT 工具库 (JJWT) -->
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.5</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>
```

任务 1-2: 清理旧代码

1. **删除或注释**第14周创建的 `UserInterceptor` (简易拦截器) 及其在 `WebConfig` 中的注册代码。我们将使用更强大的 Spring Security 过滤器链来替代它。
 2. 保留 `User` 实体和 `UserRepository` 。
-

第二部分：核心安全组件实现（1.5学时）

任务 2-1：完善 User 实体

确保 `User` 实体中包含角色字段（如果之前没有），并实现密码加密存储的准备。

```
@Entity
@Table(name = "users")
@Data
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String username;

    @Column(nullable = false)
    private String password; // 存储加密后的哈希值

    private String email;

    private String role = "ROLE_USER"; // 默认角色
}
```

任务 2-2：编写 JWT 工具类 (JwtUtil)

创建一个工具类 `JwtUtil`，用于生成和解析 Token。

```

@Component
public class JwtUtil {
    // 密钥，实际生产中应从配置文件读取
    private final String SECRET_KEY = "MySuperSecretKeyForPkmSystemDoNotLeakIt";
    private final long EXPIRATION_TIME = 86400000; // 24小时

    private Key getSigningKey() {
        return Keys.hmacShaKeyFor(SECRET_KEY.getBytes(StandardCharsets.UTF_8));
    }

    // 生成 Token
    public String generateToken(String username) {
        return Jwts.builder()
            .setSubject(username)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + EXPIRATION_TIME))
            .signWith(getSigningKey(), SignatureAlgorithm.HS256)
            .compact();
    }

    // 验证 Token 并提取用户名
    public String extractUsername(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(getSigningKey())
            .build()
            .parseClaimsJws(token)
            .getBody()
            .getSubject();
    }

    // 校验 Token 是否有效
    public boolean validateToken(String token) {
        try {
            Jwts.parserBuilder().setSigningKey(getSigningKey()).build().parseClaimsJws(token);
            return true;
        } catch (Exception e) {
            return false;
        }
    }
}

```

任务 2-3：实现 UserDetailsService

Spring Security 需要通过这个接口加载用户信息。

```

@Service
public class CustomUserDetailsService implements UserDetailsService {

    @Autowired
    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found: " +
username));

        // 将我们的 User 实体转换为 Spring Security 的 UserDetails
        return org.springframework.security.core.userdetails.User
            .withUsername(user.getUsername())
            .password(user.getPassword())
            .authorities(user.getRole())
            .build();
    }
}

```

任务 2-4: 编写 JWT 认证过滤器 (JwtAuthenticationFilter)

这是连接 JWT 和 Spring Security 的桥梁。

```

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    @Autowired
    private JwtUtil jwtUtil;
    @Autowired
    private CustomUserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse
response, FilterChain chain)
        throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");

        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            String token = authHeader.substring(7);
            if (jwtUtil.validateToken(token)) {
                String username = jwtUtil.extractUsername(token);
                UserDetails userDetails =
userDetailsService.loadUserByUsername(username);

                // 构建认证对象
                UsernamePasswordAuthenticationToken authentication =
                    new UsernamePasswordAuthenticationToken(userDetails, null,
userDetails.getAuthorities());

                // 放入安全上下文
                SecurityContextHolder.getContext().setAuthentication(authentication);
            }
        }
        chain.doFilter(request, response);
    }
}

```

任务 2-5：安全配置类 (SecurityConfig)

组装所有组件，配置路由规则。

```

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Autowired
    private JwtAuthenticationFilter jwtAuthenticationFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf().disable() // 禁用 CSRF

            .sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS) // 禁用 Session

            .and()
            .authorizeRequests()
            .antMatchers("/api/auth/**").permitAll() // 放行登录注册接口
            .anyRequest().authenticated(); // 其他接口需认证

        // 添加 JWT 过滤器
        http.addFilterBefore(jwtAuthenticationFilter,
            UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(); // 使用 BCrypt 加密
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration
config) throws Exception {
        return config.getAuthenticationManager();
    }
}

```

第三部分：业务逻辑适配（1学时）

任务 3-1：实现注册与登录接口

创建 `AuthController`。

```

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired
    private AuthenticationManager authenticationManager;
    @Autowired
    private UserRepository userRepository;
    @Autowired
    private PasswordEncoder passwordEncoder;
    @Autowired
    private JwtUtil jwtUtil;

    // 注册
    @PostMapping("/register")
    public ResponseEntity<?> register(@RequestBody User user) {
        if (userRepository.findByUsername(user.getUsername()).isPresent()) {
            return ResponseEntity.badRequest().body("Username is already taken!");
        }
        // 加密密码
        user.setPassword(passwordEncoder.encode(user.getPassword()));
        userRepository.save(user);
        return ResponseEntity.ok("User registered successfully!");
    }

    // 登录
    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody User loginRequest) {
        try {
            // 验证用户名密码
            authenticationManager.authenticate(
                new UsernamePasswordAuthenticationToken(loginRequest.getUsername(),
                    loginRequest.getPassword())
            );
        } catch (BadCredentialsException e) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Invalid
            username or password");
        }

        // 生成 Token
        String token = jwtUtil.generateToken(loginRequest.getUsername());
        return ResponseEntity.ok(Collections.singletonMap("token", token));
    }
}

```

任务 3-2: 重构 UserContext

修改第14周的 `UserContext`，不再从 `ThreadLocal` 取值，而是从 `SecurityContext` 获取当前用户。这样 `NoteService` 不需要做任何修改就能无缝对接安全框架。

```
@Component
public class UserContext {

    @Autowired
    private UserRepository userRepository;

    /**
     * 获取当前登录用户的 ID
     * 注意：这里不再是静态方法，建议注入使用，或者通过 ApplicationContext 获取 Bean
     */
    public Long getCurrentUserId() {
        Authentication authentication =
            SecurityContextHolder.getContext().getAuthentication();
        if (authentication == null || !authentication.isAuthenticated() ||
            "anonymousUser".equals(authentication.getPrincipal())) {
            throw new RuntimeException("User not authenticated");
        }

        // 从 UserDetails 中获取用户名，再查数据库获取 ID
        // 优化建议：可以让 UserDetails 直接包含 ID，避免查库
        UserDetails userDetails = (UserDetails) authentication.getPrincipal();
        return userRepository.findByUsername(userDetails.getUsername())
            .map(User::getId)
            .orElseThrow(() -> new RuntimeException("User not found"));
    }
}
```

注意：如果之前的 `UserContext` 是全静态方法的工具类，你需要将其改为 *Spring Bean*，或者在静态方法内部通过 `SpringContextUtil` 获取 `UserRepository`。为了简单起见，建议将 `NoteService` 中调用 `UserContext.getCurrentUserId()` 的地方改为注入 `UserContext` *Bean* 调用。

第四部分：验证与测试 (1学时)

任务 4-1：使用 Postman 测试注册

1. URL: POST `http://localhost:8080/api/auth/register`
2. Body: `{"username": "testuser", "password": "password123"}`
3. 预期: 返回注册成功，数据库中密码字段是乱码（加密后的 Hash）。

任务 4-2：使用 Postman 测试登录

1. URL: POST `http://localhost:8080/api/auth/login`
2. Body: `{"username": "testuser", "password": "password123"}`
3. 预期: 返回 JSON `{"token": "eyJhbGciOiJIUzI1Ni..."}`。

任务 4-3: 测试受保护的接口

1. 不带 Token 访问: GET `http://localhost:8080/api/notes` -> 403 Forbidden。
2. 带 Token 访问:
 - 在 Postman 的 Auth 标签页选择 Bearer Token。
 - 粘贴刚才获取的 Token。
 - 发送请求 -> 200 OK (返回该用户的笔记列表)。

实验总结

本周实验完成了 PKM 系统从“裸奔”到“全副武装”的蜕变。我们集成了业界标准的 Spring Security + JWT 方案，实现了：

1. 无状态认证：服务器不再存储 Session，适合分布式扩展。
2. 密码安全：数据库中不再存储明文密码。
3. 透明的上下文切换：业务层代码（NoteService）几乎不需要修改，通过重构底层 Context 获取逻辑，平滑过渡到了安全框架。

提交

```
git commit
git tag -a week15-release -m "v5.0 release"
git push
```

git提交，设置tag week15_release_1_0，然后推送到服务器

课后思考

1. JWT Token 一旦签发，在过期前都是有效的。如果用户修改了密码或被管理员封禁，如何让旧 Token 立即失效？（提示：黑名单机制 / 版本号机制）
2. 目前的 `UserContext` 每次获取 ID 都要查数据库，如何优化？（提示：自定义 `UserDetails` 实现类，直接在 Token 解析阶段将 ID 放入 `Authentication` 对象中）